Lab 3: Architectural Pattern Selection – Justification

Self-Service Coffee Kiosk System

Architecture Selection: We chose the Layered Architecture for the Self-Service Coffee Kiosk System.

Reason 1 – Clear separation of concerns

The kiosk has three distinct concerns: a touch-screen presentation surface, order/payment business logic, and persistent menu/pricing data. A layered design (Presentation, Business, Data) maps directly onto these concerns. The User Interface Component only handles touch-screen rendering and input, the Order Manager Component owns the ordering workflow, and the Database Component isolates all menu and pricing storage. This keeps each layer simple, independently testable, and easy for a small development team to understand and maintain – well suited to a single-purpose kiosk application rather than a large distributed system.

Reason 2 – Matches the scale and deployment model of a single kiosk

Each kiosk is a single, self-contained unit of hardware (touch screen, card reader, receipt printer) running one application instance. There is no requirement for independent scaling of individual functions or multi-team ownership, so the operational overhead of a microservices style is unnecessary. A layered monolith is simpler to build, deploy, and support on embedded kiosk hardware, while still giving each layer a well-defined interface (Order Interface, Payment Interface, Receipt Interface, Database Interface) so components can be replaced or upgraded independently.

Security Advantage

Layering enforces that the Presentation Layer never talks to the Data Layer directly – all requests must pass through the Order Manager Component in the Business Layer. This means credit-card payment requests and menu/pricing data are never exposed straight to the touch-screen UI; the Payment Service Component is the only component that communicates with the card processor, and the Database Component is only reachable through the Order Manager's controlled interface. This reduces the attack surface and keeps sensitive payment handling isolated from the user-facing layer.

Performance Benefit

Because each layer exposes a narrow, well-defined interface, the Order Manager Component can cache frequently requested menu and pricing data from the Database Component instead of querying it for every screen refresh. Localizing hardware-specific work (touch input, receipt printing) to dedicated components also keeps the UI thread responsive, ensuring the touch-screen ordering flow stays fast even while a receipt is printing or a payment is being authorized in the background.